

Voice Based Examination System - Bugs & Architecture Analysis

This document summarizes the major architectural issues, bugs, risks, and improvement areas identified in the Voice Based Examination System project.

1. Whisper Not Actually Integrated

The project includes the full whisper.cpp source code, but the application likely still relies on browser-based speech recognition APIs such as webkitSpeechRecognition.

This means Whisper exists in the repository but is not truly connected to the backend transcription pipeline.

Missing components likely include:

- Audio streaming to Whisper
- Backend transcription routes
- Model loading
- .bin model management

2. Fragmented Voice Engine

The frontend voice system is heavily modularized across many files:

recognition.js, commands.js, command_matcher.js, transcript.js, lifecycle.js, state.js, autosave.js, and recovery.js.

This architecture is strong conceptually, but likely introduces:

- Race conditions
- Event conflicts
- Duplicate states
- Microphone lifecycle issues

3. Speech Recognition Auto-Stop

Browser speech recognition APIs frequently stop because of silence, browser throttling, or timeout events.

Without a safe restart mechanism, the exam flow may break entirely.

Potential symptoms:

- Recognition stopping randomly
- Infinite restart loops
- Multiple recognition instances
- Browser crashes

4. Duplicate Submission Risk

Autosave and manual submission may both write answers simultaneously.

Without:

- Submission locks

- Idempotency
 - Version control
- duplicate answer records can appear.

5. Missing Recovery System

Crash recovery and reconnect handling appear incomplete.

Possible issues:

- Browser refresh loses answers
- Session state mismatch
- Network interruptions corrupt progress

6. State Synchronization Problems

Frontend state appears distributed across multiple files instead of using a centralized store.

Likely issues:

- Question mismatch
- Timer mismatch
- Transcript mismatch
- Backend/frontend inconsistency

7. Timer Desynchronization

Frontend and backend timers may drift apart.

Potential consequences:

- Unexpected auto-submit
- Students manipulating timers
- Inconsistent exam timing

8. Session Security Weaknesses

Possible security risks include:

- Weak authorization checks
- Session fixation
- Route protection gaps
- Manual role validation

9. Role Authorization Gaps

Routes may not consistently use decorators such as:

@admin_required

@candidate_required

This can allow unauthorized dashboard access.

10. Database Transaction Safety

Critical exam operations should be atomic.

Without transaction handling:

- Partial saves
- Answer corruption
- Inconsistent submissions

may occur.

11. Weak Anti-Cheat Layer

Browser-based anti-cheat systems are inherently limited.

Current checks likely include:

- Tab switching
- Fullscreen detection

But users can still bypass these using:

- Another device
- Split-screen setups
- Disabled JavaScript

12. Missing Offline Queue

When the internet disconnects, answers may fail to save.

A retry synchronization queue is required for production-grade reliability.

13. Fragile Command Matching

Speech variations can break command parsing.

Example:

“next question”

“neck question”

“next cautions”

Without fuzzy matching, false positives and failed commands are common.

14. TTS Overlapping Recognition

Text-to-Speech may be captured again by the recognition engine itself.
Without proper pause/resume handling:
• Voice loops
• Self-triggering commands
may occur.

15. Multiple Event Listener Bugs

Repeatedly attaching listeners can cause:

- Double saves
- Duplicate navigation
- Multiple transcripts
- Commands executing twice

16. Memory Leaks During Long Exams

Continuous recognition sessions may leak memory if:

- Event listeners are not removed
- Intervals are not cleared
- Transcript history grows endlessly

17. Backend/Frontend Source-of-Truth Conflict

Frontend and backend answer states may differ.

Example:

Frontend answer = B

Backend saved answer = A

This causes severe data consistency problems.

18. AI Grading Instability

If AI grading exists, the same answer may receive different scores.

This requires:

- Rubric enforcement
- Deterministic evaluation
- Moderation layers

19. Missing Test Coverage

The tests folder appears mostly empty.

Missing:

- Integration tests
- Stress tests
- Concurrency tests
- Voice engine tests

20. No Real-Time Monitoring

The invigilator system likely lacks:

- WebSocket support
- Real-time sync
- Live candidate monitoring

21. Scalability Risks

The system likely works for small-scale demos but may fail under high concurrency.

Potential bottlenecks:

- Database overload
- Autosave flooding
- Session collisions
- Speech-processing lag

22. Biggest Architectural Issue

The frontend is advanced and feature-rich, but the backend reliability layer is incomplete.

The project still lacks:

- Reliable recovery
- Transaction safety
- Offline synchronization
- Robust concurrency handling
- Proper Whisper integration
- Security hardening

23. Highest Priority Fixes

1. Proper Whisper integration
2. Duplicate submission prevention
3. Recovery/reconnect system
4. Stable speech recognition restart flow
5. Timer synchronization
6. Session consistency
7. Offline save queue
8. Event listener cleanup
9. Transaction safety

24. Positive Aspects

Despite the issues, the project architecture is significantly stronger than most student projects because it demonstrates:

- Layered architecture
- Service-oriented backend design
- Modular voice engine
- Recovery and autosave thinking
- Anti-cheat awareness
- Advanced frontend structure

The project already resembles a real product architecture more than a standard CRUD-based college project.